

END-TERM PROJECT - 02 REPORT

ON

FLAPPY BIRD GAME

B.Tech. Computer Science and Engineering

Semester: 6th

Section: 23412C1

Course Title & Code: Game Programming With C++ (CSE 3545)

Submitted By:

Student Name: Swayam Swarup Jethi

Registration No: 2341010085

Guided By: Prof. Mohammad Maksood Akhter

Academic Year 2026



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING AND TECHNOLOGY, INSTITUTE OF
TECHNICAL EDUCATION AND RESEARCH
SIKSHA 'O' ANUSANDHAN (DEEMED TO BE) UNIVERSITY
BHUBANESWAR, ODISHA, INDIA

ABSTRACT

This project focuses on the development of a simple 2D Flappy Bird Game using C++ and SFML (Simple and Fast Multimedia Library). The game allows the player to control a bird that moves through gaps between pipes while avoiding collisions. It is designed using Object-Oriented Programming concepts and includes features such as gravity-based movement, collision detection, score tracking, difficulty levels, pause/resume functionality, sound effects, and high score storage using file handling. The project provides practical experience in game development, event handling, graphics rendering, and multimedia programming using modern C++ techniques.

Submitted By

Guided By

TABLE OF CONTENTS

Section No.	Topic	Page No.
01	Introduction	1
02	Problem Statement	2
03	Objectives of the Project	3
04	Tools and Technologies Used	4
05	Methodology	5
06	Results Analysis and Screenshots	6-7
07	Logical Understanding	8
08	Conclusion	9
09	Future Scopes and Application	10
	References	11
	Annexure	12-20

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Main Menu Screen	6
Figure 2	Gameplay Window	6
Figure 3	Pause Screen	7
Figure 4	Game Over Screen	7

01. Introduction

The Flappy Bird Game is a simple 2D side-scrolling arcade game developed using C++ and SFML (Simple and Fast Multimedia Library). In this game, the player controls a bird that continuously falls due to gravity and must pass through gaps between moving pipes while avoiding collisions. The project is designed to understand the fundamentals of game development and multimedia programming using modern C++ techniques.

The game is developed using Object-Oriented Programming concepts such as classes, encapsulation, and modular design. It includes features like bird flap mechanics, random pipe generation, collision detection, score tracking, high score storage, difficulty levels, pause/resume functionality, and sound effects. SFML graphics and audio modules are used to provide smooth gameplay and an interactive user experience.

02. Problem Statement

The problem addressed in this project is to design and implement a simple 2D Flappy Bird Game using C++ and SFML.

The game is developed to demonstrate real-time game mechanics where the player controls a bird that must pass through moving pipes while avoiding collisions.

The project uses concepts such as object-oriented programming, event handling, animation, graphics rendering, sound integration, and file handling to create an interactive gaming application.

03. Objectives of the Project

The objectives of this project are defined to provide a clear direction for the design and development of the Flappy Bird Game. The objectives of this practical project are:

- To develop a simple 2D side-scrolling game using C++ and SFML.
- To implement Object-Oriented Programming concepts using classes such as Bird and Pipe.
- To implement gravity-based bird movement and flap mechanics.
- To generate pipes at random positions with gaps for navigation.
- To implement real-time collision detection between the bird and obstacles.
- To design a score tracking and high score storage system using file handling.
- To implement different difficulty levels for gameplay.
- To integrate sound effects and graphical textures using SFML.
- To implement pause, resume, and restart functionalities.
- To improve understanding of game development concepts, event handling, and multimedia programming using C++.

04. Tools and Technologies Used

The following tools and technologies were used in this project:

Programming Language: C++

Graphics and Audio Library: SFML (Simple and Fast Multimedia Library)

Compiler: G++

Development Environment: Visual Studio Code

Operating System: Ubuntu WSL (Windows Subsystem for Linux)

These tools provide a suitable environment for writing, compiling, and executing C++ graphical game applications.

05. Methodology

The methodology followed during the development of this project includes:

- **Understanding the Problem:** Identifying the game requirements, player controls, collision system, and expected gameplay output.
- **Game Design:** Designing the bird mechanics, pipe generation system, scoring system, and difficulty levels.
- **Program Development:** Writing the game logic and graphical interface using C++ and SFML.
- **Object-Oriented Implementation:** Creating separate classes such as Bird and Pipe for modular and reusable code structure.
- **Compilation and Execution:** Compiling and running the program using the G++ compiler in Ubuntu WSL.
- **Testing and Debugging:** Detecting errors related to movement, collisions, graphics, and audio, and improving gameplay accuracy.
- **Final Output:** Ensuring smooth gameplay, proper scoring, sound effects, and correct game functionality.

06. Results Analysis and Screenshots

This section describes the results obtained after implementing the Flappy Bird Game using C++ and SFML. The game was successfully executed and produced the expected graphical and interactive output.

The implementation demonstrates the correct use of object-oriented programming concepts, collision detection, real-time rendering, event handling, and multimedia integration.

Output Screenshots:



Figure 1 : Main Menu Screen

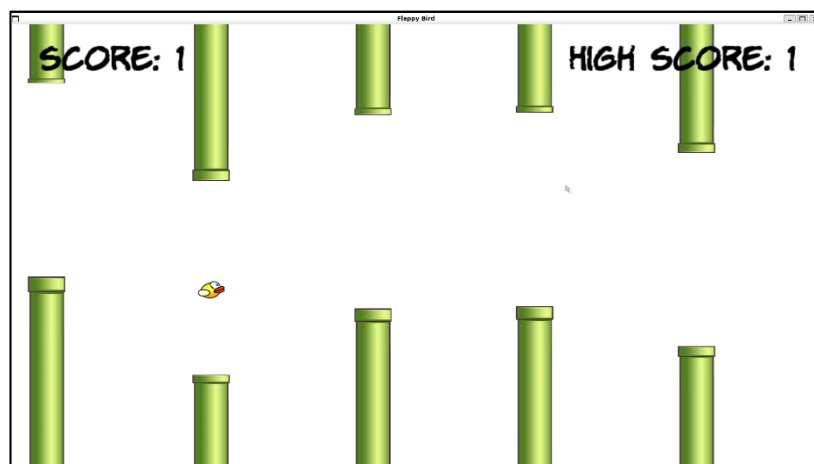


Figure 2 : Gameplay Screen

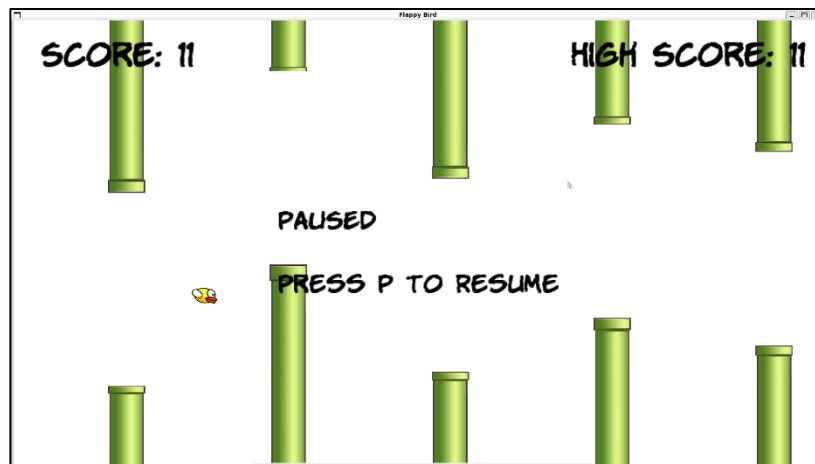


Figure 3 : Pause Screen

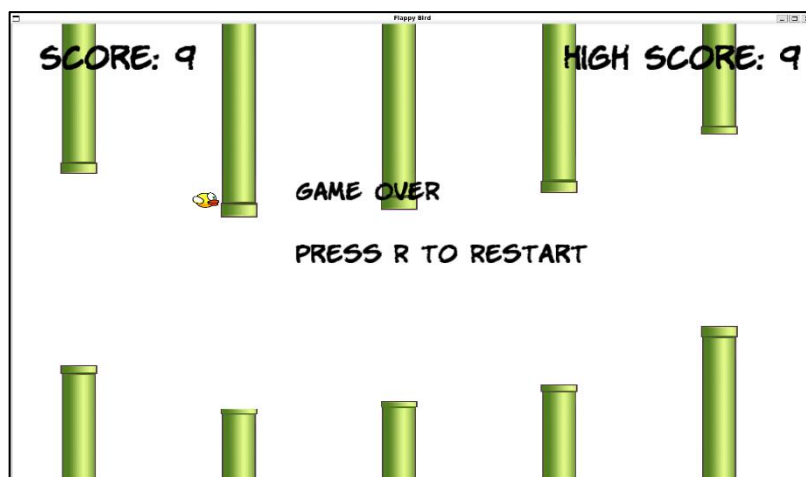


Figure 4 : Game Over Screen

07. Logical Understanding

The logic of the program is based on:

- Handling keyboard input from the player for bird movement and game controls.
- Processing game mechanics such as gravity, flap movement, pipe generation, collision detection, and score calculation using conditions and loops.
- Applying Object-Oriented Programming concepts through classes such as Bird and Pipe for modular and organized coding.
- Using a continuous game loop for updating graphics, movement, collision checks, and rendering output in real time.
- Displaying the graphical interface, score, high score, and game status clearly using SFML graphics components.

This project helps in understanding game loop execution, event handling, collision detection, multimedia programming, and logical flow control in C++ programming.

.

08. Conclusion

In conclusion, this practical project was successfully completed using the C++ programming language and SFML library. The project strengthened the understanding of fundamental concepts such as object-oriented programming, event handling, collision detection, real-time rendering, multimedia integration, and game loop execution.

The implementation of bird movement, pipe generation, scoring system, difficulty levels, and sound effects provided practical experience in developing interactive graphical applications. The project also improved problem-solving abilities and programming skills related to game development.

09. Future Scopes and Application

The future improvements and applications of this project include:

- Adding advanced gameplay features such as multiple bird characters, animated backgrounds, power-ups, and achievements.
- Improving game performance and efficiency using advanced C++ programming concepts and optimized rendering techniques.
- Extending the game with additional levels.
- Creating a more user-friendly graphical interface with improved textures, menus, animations, and visual effects.
- Enhancing file handling features for storing player profiles, settings, game statistics, and leaderboard data.

Game development using C++ and SFML has applications in multimedia software development, simulation systems, educational games, and interactive entertainment systems.

References

- (1) SFML Official Documentation – <https://www.sfml-dev.org/documentation/>
- (2) C++ Reference – <https://cplusplus.com/>
- (3) SFML Graphics Module Tutorial – <https://www.sfml-dev.org/tutorials/>
- (4) Visual Studio Code Documentation – <https://code.visualstudio.com/docs>
- (5) GNU G++ Compiler Documentation – <https://gcc.gnu.org/>

Annexure

The following section includes the code of the game including the explanation for different portions.

main.cpp

```
#include <SFML/Graphics.hpp>
#include <SFML/Audio.hpp>
#include <iostream>
#include <vector>
#include <fstream>
#include <ctime>

#include "Bird.h"
#include "Pipe.h"

using namespace sf;
using namespace std;

const int WIDTH = 900;
const int HEIGHT = 600;

enum GameState
{
    MENU,
    PLAYING,
    PAUSED,
    GAMEOVER
};

enum Difficulty
{
    EASY,
    MEDIUM,
    HARD
};

int main()
{
    srand(time(0));
    RenderWindow window(VideoMode(WIDTH, HEIGHT), "Flappy Bird");

    window.setFramerateLimit(60);
    SoundBuffer flapBuffer;
    SoundBuffer hitBuffer;

    flapBuffer.loadFromFile("assets/audio/flap.wav");
    hitBuffer.loadFromFile("assets/audio/hit.wav");

    Sound flapSound;
    Sound hitSound;

    flapSound.setBuffer(flapBuffer);
    hitSound.setBuffer(hitBuffer);

    Font font;
    font.loadFromFile("assets/fonts/KOMIKAP_.ttf");

    Text scoreText;
    Text highScoreText;

    scoreText.setFont(font);
    scoreText.setCharacterSize(40);
    scoreText.setFill(Color::Black);
```



```

scoreText.setPosition(30, 20);

highScoreText.setFont(font);
highScoreText.setCharacterSize(40);
highScoreText.setFillColor(Color::Black);
highScoreText.setPosition(620, 20);

Text infoText;

infoText.setFont(font);
infoText.setCharacterSize(30);
infoText.setFillColor(Color::Black);
infoText.setPosition(250, 180);

Bird bird;
vector<Pipe> pipes;

Clock pipeClock;
GameState gameState = MENU;

int score = 0;
int highScore = 0;

float gapSize = 180;
float pipeSpeed = 4;
float spawnInterval = 1.0f;

bool oscillation = false;

ifstream inFile("highscore.txt");

if (inFile.is_open())
{
    inFile >> highScore;
}

inFile.close();

while (window.isOpen())
{
    Event event;

    while (window.pollEvent(event))
    {
        if (event.type == Event::Closed)
        {
            window.close();
        }
        if (event.type == Event::KeyPressed)
        {
            if (gameState == MENU)
            {
                if (event.key.code == Keyboard::Num1)
                {
                    gapSize = 250;
                    pipeSpeed = 3;
                    oscillation = false;
                    gameState = PLAYING;
                }

                if (event.key.code == Keyboard::Num2)
                {
                    gapSize = 180;
                    pipeSpeed = 4;
                    oscillation = false;
                    gameState = PLAYING;
                }
            }
        }
    }
}

```

```

        if (event.key.code == Keyboard::Num3)
        {
            gapSize = 140;
            pipeSpeed = 5;
            oscillation = true;
            gameState = PLAYING;
        }
    }
    else if (gameState == PLAYING)
    {
        if (event.key.code == Keyboard::Space)
        {
            bird.flap();
            flapSound.play();
        }

        if (event.key.code == Keyboard::P)
        {
            gameState = PAUSED;
        }
    }
    else if (gameState == PAUSED)
    {
        if (event.key.code == Keyboard::P)
        {
            gameState = PLAYING;
        }
    }
    else if (gameState == GAMEOVER)
    {
        if (event.key.code == Keyboard::R)
        {
            score = 0;
            pipes.clear();
            bird.reset();
            gameState = MENU;
        }
    }
}

if (gameState == PLAYING)
{
    bird.update();
    if (pipeClock.getElapsedTime().asSeconds() > spawnInterval)
    {
        float margin = 60.f;
        float minGapY = margin;
        float maxGapY = HEIGHT - gapSize - margin;

        if (maxGapY < minGapY)
        {
            maxGapY = minGapY;
        }
        float gapY = minGapY +
            (static_cast<float>(rand()) / RAND_MAX) *
            (maxGapY - minGapY);

        pipes.push_back(
            Pipe(WIDTH, gapY, gapSize, pipeSpeed, HEIGHT));

        pipeClock.restart();
    }
    for (int i = 0; i < pipes.size(); i++)
    {
        pipes[i].update(oscillation);
        if (!pipes[i].passed &&

```

```

        pipes[i].topPipe.getPosition().x < 200)
    {
        score++;

        pipes[i].passed = true;
    }
    if (pipes[i].checkCollision(bird.getBounds()))
    {
        gameState = GAMEOVER;

        hitSound.play();
    }
}
while (!pipes.empty() &&
        pipes.front().offScreen())
{
    pipes.erase(pipes.begin());
}
if (bird.getY() > HEIGHT)
{
    gameState = GAMEOVER;

    hitSound.play();
}
if (score > highScore)
{
    highScore = score;
    ofstream outFile("highscore.txt");
    outFile << highScore;
    outFile.close();
}
}
window.clear(Color(Color::White));
for (int i = 0; i < pipes.size(); i++)
{
    pipes[i].draw(window);
}
bird.draw(window);
scoreText.setString(
    "Score: " + to_string(score));

highScoreText.setString(
    "High Score: " + to_string(highScore));

window.draw(scoreText);
window.draw(highScoreText);
if (gameState == MENU)
{
    infoText.setString(
        "FLAPPY BIRD\n\n"
        "Press 1 : Easy\n"
        "Press 2 : Medium\n"
        "Press 3 : Hard"
    );
    FloatRect textRect =
        infoText.getLocalBounds();
    infoText.setOrigin(
        textRect.left + textRect.width / 2.0f,
        textRect.top + textRect.height / 2.0f
    );

    infoText.setPosition(
        WIDTH / 2.0f,
        HEIGHT / 2.0f
    );

    window.draw(infoText);
}

```

```

}
if (gameState == PAUSED)
{
    infoText.setString(
        "PAUSED\n\n"
        "Press P to Resume"
    );

    FloatRect textRect =
        infoText.getLocalBounds();

    infoText.setOrigin(
        textRect.left + textRect.width / 2.0f,
        textRect.top + textRect.height / 2.0f
    );

    infoText.setPosition(
        WIDTH / 2.0f,
        HEIGHT / 2.0f
    );

    window.draw(infoText);
}

if (gameState == GAMEOVER)
{
    infoText.setString(
        "GAME OVER\n\n"
        "Press R to Restart");
    window.draw(infoText);
}

window.display();
}
return 0;
}

```

Explanation :

Handles window creation, game loop, and event input for all states.

Manages score, high score file I/O, and difficulty settings.

Spawns and updates pipes, checks collisions, and triggers sounds.

Renders the bird, pipes, and UI text for menu/pause/game over.

Bird.h

```

#ifndef BIRD_H
#define BIRD_H

#include <SFML/Graphics.hpp>
using namespace sf;
class Bird{
private:
    Texture texture;
    Sprite spriteBird;
    float velocity;

public:
    Bird();

```

```

    void flap();
    void update();
    void reset();
    void draw(RenderWindow &window);
    FloatRect getBounds();
    float getY();
};

#endif

```

Explanation :

Declares the Bird class with texture, sprite, and motion state.

Exposes methods for flap, update, reset, draw, and bounds checks.

Provides accessors used by the main loop for collision and limits.

Keeps the bird logic encapsulated from the rest of the game.

Bird.cpp

```

#include "Bird.h"
using namespace sf;
Bird::Bird(){
    texture.loadFromFile("assets/graphics/bird.png");
    spriteBird.setTexture(texture);
    spriteBird.setScale(0.2f, 0.2f);
    spriteBird.setPosition(200, 300);
    velocity = 0;
}

void Bird::flap(){
    velocity = -8.5f;
}

void Bird::update(){
    velocity += 0.45f;
    spriteBird.move(0, velocity);
    spriteBird.setRotation(velocity * 3);
    if (spriteBird.getPosition().y < 0){
        spriteBird.setPosition(spriteBird.getPosition().x, 0);
        velocity = 0;
    }
}

void Bird::reset(){
    spriteBird.setPosition(200, 300);
    velocity = 0;
    spriteBird.setRotation(0);
}

void Bird::draw(sf::RenderWindow &window){
    window.draw(spriteBird);
}

sf::FloatRect Bird::getBounds(){
    return spriteBird.getGlobalBounds();
}

```

```
float Bird::getY(){
    return spriteBird.getPosition().y;
}
```

Explanation :

Loads bird texture, sets initial scale and position.

Implements gravity and flap impulse to control vertical motion.

Rotates the sprite based on velocity for visual feedback.

Clamps movement at the top and resets state on restart.

Pipe.h

```
#ifndef PIPE_H
#define PIPE_H
#include <SFML/Graphics.hpp>
using namespace sf;
class Pipe{
private:
    Texture texture;

public:
    Sprite topPipe;
    Sprite bottomPipe;
    bool passed;
    float speed;
    Pipe(float x, float gapY, float gapSize, float spd, float screenH);
    void update(bool oscillate);
    void draw(RenderWindow &window);
    bool offScreen();
    bool checkCollision(FloatRect birdBounds);
};

#endif
```

Explanation :

Declares the 'Pipe' class with shared texture and sprites.

Stores speed, pass state, and exposes update/draw helpers.

Provides collision check and off-screen detection for cleanup.

Keeps pipe logic modular for easy spawning and management.

Pipe.cpp

```
#include "Pipe.h"
#include <iostream>

using namespace sf;
namespace{
    Texture pipeTexture;
    bool loaded = false;
}

Pipe::Pipe(float x, float gapY, float gapSize, float spd, float screenH){
    pipeTexture.loadFromFile("assets/graphics/pipe.png");
    topPipe.setTexture(pipeTexture);
    bottomPipe.setTexture(pipeTexture);

    float screenHeight = screenH;
    float scaleX = 0.8f;

    Vector2u texSize = pipeTexture.getSize();
    float textureHeight = static_cast<float>(texSize.y);
    if (textureHeight < 1.f){
        textureHeight = 1.f;
    }

    float topHeight = gapY;
    float bottomHeight = screenHeight - (gapY + gapSize);

    if (topHeight < 0.f){
        topHeight = 0.f;
    }
    if (bottomHeight < 0.f){
        bottomHeight = 0.f;
    }

    topPipe.setScale(scaleX, -topHeight / textureHeight);
    topPipe.setPosition(x, gapY);
    bottomPipe.setScale(scaleX, bottomHeight / textureHeight);
    bottomPipe.setPosition(x, gapY + gapSize);
    speed = spd;
    passed = false;
}

void Pipe::update(bool oscillate){
    topPipe.move(-speed, 0);
    bottomPipe.move(-speed, 0);
}

void Pipe::draw(RenderWindow &window){
    window.draw(topPipe);
    window.draw(bottomPipe);
}

bool Pipe::offScreen(){
    float width = topPipe.getGlobalBounds().width;
    return (topPipe.getPosition().x + width) < 0;
}
```

```
bool Pipe::checkCollision(FloatRect birdBounds){  
    return birdBounds.intersects(topPipe.getGlobalBounds()) ||  
        birdBounds.intersects(bottomPipe.getGlobalBounds());  
}
```

Explanation :

Loads the pipe texture once and reuses it across all pipes.

Builds top and bottom pipes by scaling to the gap position.

Moves pipes left each frame and reports when off screen.

Detects collisions against the bird's bounding rectangle.